

Aula 4 — PWA, Service Workers & Offline-First

"A web instalável: offline, rápida e nativa o suficiente"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 01/07/2026

Recap da Aula 3

- Flutter: widgets, Stateless vs Stateful, Navigator
- Compose Multiplatform — compartilhando UI entre Android e iOS
- **KMP**: próxima aula (Aula 5) — hoje focamos em PWA

Atividade 3 — entregue?

Agenda da aula (3h30)

Bloco 1 (90min) — PWA: o que é, manifest, ciclo de vida SW

→ Anatomia · install/activate/fetch · cache strategies

Intervalo (30min)

Bloco 2 (60min) — Workbox + IndexedDB + Background Sync

→ Workers · Lighthouse CI · hands-on A7

Fechamento (30min) — Atividade 7 ao vivo + dúvidas

Objetivos de aprendizagem

- Compreender ciclo de vida de Service Workers (install, activate, fetch)
- Aplicar cache strategies com Workbox (NetworkFirst, CacheFirst, StaleWhileRevalidate)
- Converter SPA em app instalável e offline-first
- Entender diferença entre Web Worker e Service Worker
- Medir qualidade PWA com Lighthouse

"PWA é só web?" — fato vs ficção

Twitter Lite: ~30% do tráfego mobile do Twitter já em PWA, com offline real.

Starbucks: PWA com pedido offline robusto.

Pinterest Lite: instalável, push, offline.

Spotify Web: virou PWA instalável em 2023.

PWA não é cosmético. Em 2026, com Background Sync + Web Push + offline + install desktop, cobre casos antes exclusivos de nativo.

Anatomia de uma PWA

Uma PWA precisa de **3 componentes**:

1. **Web App Manifest** (`manifest.json`) — metadados (nome, ícones, cores, display)
2. **Service Worker** — interceptador de network, cache, offline
3. **HTTPS** — origem segura obrigatória

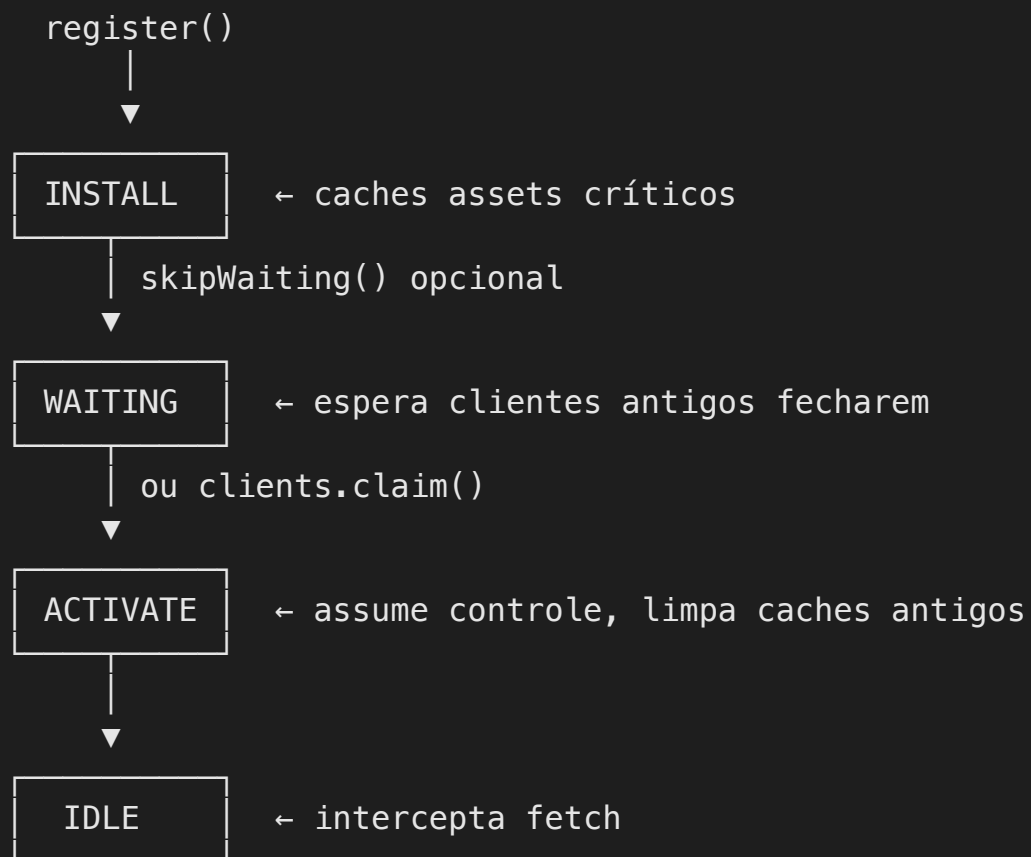
Bonus para "instalável":

- Ícones em múltiplos tamanhos
- `start_url`, `display: standalone`
- Service Worker com `fetch handler`

Web App Manifest

```
{
  "name": "Minha PWA",
  "short_name": "MinhaApp",
  "description": "Descrição curta da app",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#003366",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```


Service Worker — lifecycle



Service Worker — install + activate

```
// sw.js
const CACHE_NAME = 'app-v3';
const PRECACHE_URLS = ['/', '/index.html', '/styles.css', '/app.js'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(PRECACHE_URLS))
  );
  self.skipWaiting(); // assume controle imediato (cuidado!)
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((names) =>
      Promise.all(
        names.filter((n) => n !== CACHE_NAME).map((n) => caches.delete(n))
      )
    )
  );
  self.clients.claim();
});
```

Cache strategies

Cache-first (estáticos)

Tenta cache; se falhar, vai pra rede; cacheia o resultado.

Network-first (dados dinâmicos)

Tenta rede com timeout; se falhar, vai pro cache.

Stale-While-Revalidate

Serve cache imediatamente (rápido) + busca rede em paralelo (atualiza próxima request).

Cache-only / Network-only

Casos específicos (assets imutáveis vs telemetria).

Cache-first — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/static/')) {  
    event.respondWith(  
      caches.match(event.request).then((cached) => {  
        return cached || fetch(event.request).then((response) => {  
          const clone = response.clone();  
          caches.open(CACHE_NAME).then((cache) => cache.put(event.request, clone));  
          return response;  
        });  
      });  
    );  
  }  
});
```

Use para JS, CSS, imagens, fonts versionadas.

Stale-While-Revalidate — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/api/feed')) {  
    event.respondWith(  
      caches.open(CACHE_NAME).then(async (cache) => {  
        const cached = await cache.match(event.request);  
        const fetchPromise = fetch(event.request).then((response) => {  
          cache.put(event.request, response.clone());  
          return response;  
        });  
        return cached || fetchPromise;  
      })  
    );  
  }  
});
```

Usuário vê algo imediatamente, próxima carga já está atualizada.

O que é o Workbox?

Workbox é uma biblioteca do Google que abstrai a API de Service Worker.

Sem Workbox, você escreve cache manual para cada rota:

```
self.addEventListener('fetch', event => {  
  event.respondWith(  
    caches.match(event.request).then(cached => cached || fetch(event.request))  
  );  
});
```

Com Workbox, você declara a estratégia:

```
registerRoute(/\.png$/, new CacheFirst());
```

Sem Workbox	Com Workbox
~150 linhas de boilerplate	~10 linhas declarativas

Workbox é para Service Workers o que o Axios é para `fetch` — você podia fazer sem, mas não vale a pena.

Workbox — abstração madura

```
import { precacheAndRoute } from 'workbox-precaching';
import { registerRoute } from 'workbox-routing';
import {
  CacheFirst,
  NetworkFirst,
  StaleWhileRevalidate,
} from 'workbox-strategies';

precacheAndRoute(self.__WB_MANIFEST);

registerRoute(/\.(?:png|jpg|svg)$/ , new CacheFirst({ cacheName: 'images' }));
registerRoute(
  /\api\/static\/ ,
  new StaleWhileRevalidate({ cacheName: 'api-static' })
);
registerRoute(/\api\/dynamic\/ , new NetworkFirst({ cacheName: 'api-dynamic' }));
```

Workbox elimina boilerplate. Use em produção.

IndexedDB — persistência local estruturada

`localStorage` é síncrono e limitado (~5MB). **IndexedDB** é a opção real.

```
import Dexie from 'dexie';

const db = new Dexie('MyAppDB');
db.version(1).stores({
  drafts: '++id, content, createdAt',
  user: '&id, name, email',
});

// Add
await db.drafts.add({ content: 'Hello', createdAt: Date.now() });

// Query
const recent = await db.drafts
  .orderBy('createdAt')
  .reverse()
  .limit(10)
  .toArray();
```

Dexie torna IndexedDB usável.

Background Sync API

Usuário fecha app sem conexão. Operação fica pendente. Quando conectividade volta, **Service Worker dispara sync event mesmo com app fechado.**

```
// Page side
async function submitOfflineSafe(message) {
  await db.outbox.add({ message, attempts: 0 });

  if ('serviceWorker' in navigator && 'SyncManager' in window) {
    const registration = await navigator.serviceWorker.ready;
    await registration.sync.register('flush-outbox');
  }
}

// sw.js
self.addEventListener('sync', (event) => {
  if (event.tag === 'flush-outbox') {
    event.waitUntil(flushOutbox());
  }
});
```

Web Worker vs Service Worker — quando usar cada um?

Web Worker — "preciso processar dados sem travar a UI"

```
// Parsing de CSV gigante sem congelar a página  
const worker = new Worker('parser.js');  
worker.postMessage(csvData);           // envia pra thread separada  
worker.onmessage = e => render(e.data); // recebe resultado
```

🎯 Casos: filtrar 50k registros, aplicar filtro em imagem, rodar modelo ML

Service Worker — "preciso interceptar rede e controlar cache"

```
// Responde a fetch do app inteiro, mesmo sem internet
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request) || fetch(event.request)
  );
});
```

🎯 Casos: carregar app offline, cachear TMDB, push notification, sync em background

Web Worker = **thread paralela** pra sua página.

Service Worker = **proxy de rede** que persiste entre sessões.

Web Workers vs Service Workers

Aspecto	Web Worker	Service Worker
Propósito	Compute pesado off-thread	Network proxy + cache
Lifecycle	Scope da página	Persistente, instalado
Escopo	Página atual	Origin/scope inteiro
API	postMessage com page	fetch , cache , push , sync
Múltiplas instâncias	1 por chamada	1 por scope

Use Web Worker para parsing JSON gigante, image processing, ML em browser.

Lighthouse CI

```
# .github/workflows/lighthouse.yml
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
```

```
// lighthouseci.json
{
  "ci": {
    "assert": {
      "assertions": {
        "categories:performance": ["error", { "minScore": 0.9 }],
        "categories:pwa": ["error", { "minScore": 0.9 }]
      }
    }
  }
}
```

PWA vs RN-Web vs Capacitor

Aspecto	PWA	RN-Web	Capacitor
Distribuição	URL ou install	URL ou install	App Store + Play
Capacidade nativa	Limitada	Limitada	Total via plugins
Tamanho bundle	Pequeno	Médio	Maior
Performance	Web	Web (com RN-style)	Nativo wrapped
Update	Instantâneo	Instantâneo	Via store

Cada um cobre nichos. Não competem diretamente.

Atividade 7 — PWA: filmes instalável + offline (15 pts)

Stack: React + Vite + vite-plugin-pwa (Workbox) + TMDB REST

O que você vai implementar:

```
src/hooks/useTmdb.ts    ← TODO 1: fetchPopularMovies() com fetch()
                        ← TODO 2: useEffect + loading/error
vite.config.ts          ← TODO 3: runtimeCaching NetworkFirst pra TMDB
```

Setup:

```
cd exercicios/07-pwa-tmdb/pratica
npm install
cp .env.example .env.local    # adicione VITE_TMDB_TOKEN
npm run dev                   # http://localhost:5173
```

Teste offline:

```
npm run build && npm run preview
# DevTools → Application → Service Workers → "Offline" → F5
```


Pitfalls comuns

- ✗ `skipWaiting()` sem cuidado — clientes abertos podem ficar inconsistentes
- ✗ **Cache infinito** — esquecer de limpar caches antigos no `activate`
- ✗ `localStorage` ao invés de `IndexedDB` — limite de 5MB e síncrono
- ✗ **Service Worker em scope errado** — não intercepta o que deveria
- ✗ **Não testar em "modo airplane"** — só descobre bug em produção

Leituras recomendadas

- Ater, T. — *Building Progressive Web Apps* (O'Reilly)
- Archibald, J. — *The Offline Cookbook* (web.dev)
- W3C — *Service Worker Specification* (parte 1 e 2)

Próxima aula — Aula 5 (data a confirmar)

KMP: Kotlin Multiplatform + BFF/GraphQL

Pré-requisito:

- Atividade 7 (PWA) entregue — prazo no Canvas
- Android Studio Quail 2026.1.1+ instalado e atualizado
- Token TMDb em mãos: developer.themoviedb.org → API → Bearer

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima aula: KMP — data a confirmar